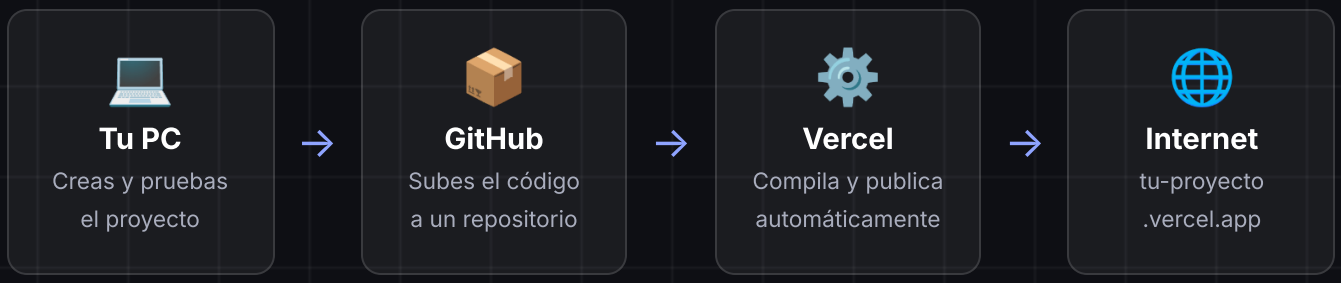
TALLER PRÁCTICO · PROGRAMACIÓN WEB

# Despliega tu proyecto React + Vite + TypeScript en Vercel

Guía paso a paso, desde cero: de tu computador a una URL pública que cualquiera puede abrir.




Universidad Popular del Cesar · Ingeniería de Sistemas

Noviembre 2026

CONTENIDO

# Índice

Toca cualquier línea para saltar a esa sección. Cada página tiene un botón ↑ **Índice** para volver aquí.


ANTES DE EMPEZAR

| | | |
|---|---|---|
| 00 | Qué necesitas y cómo funciona el despliegue | Requisitos y mapa general |

FASE 1 · EN TU COMPUTADOR

| | | |
|---|---|---|
| 01 | Verificar Node.js, npm y Git | 2 min |
| 02 | Crear el proyecto con Vite | 3 min · omite si ya tienes uno |
| 03 | Ejecutarlo en local | 2 min |
| 04 | Probar el build de producción | 3 min · el paso que más se olvida |
| 05 | Dejar el proyecto listo para desplegar | .gitignore, rutas, variables |

FASE 2 · GITHUB

| | | |
|---|---|---|
| 06 | Crear el repositorio en GitHub | 2 min |
| 07 | Subir el código con Git | 3 min |

FASE 3 · VERCEL

| | | |
|---|---|---|
| 08 | Crear la cuenta en Vercel | 2 min |
| 09 | Importar el repositorio | 2 min |
| 10 | Configurar y desplegar | 3 min |
| 11 | Ver tu sitio publicado | 1 min |
| 12 | Actualizar el sitio con cada cambio | Despliegue continuo |

EXTRAS

| | | |
|---|---|---|
| A | Alternativa: desplegar con Vercel CLI | Sin pasar por GitHub |
| B | Errores comunes y cómo solucionarlos | Léelo si algo falla |
| C | Resumen de comandos y checklist final | Una página para imprimir |


**¿YA TIENES TU PROYECTO HECHO?**

Empieza en el Paso 1 para verificar herramientas y salta directo al Paso 4. El Paso 2 solo es para quien arranca desde cero.

Las pantallas son ilustraciones; los menús de GitHub y Vercel pueden cambiar levemente. Si algo no coincide, busca el texto más parecido.

PASO 00 · ANTES DE EMPEZAR

# Qué necesitas y cómo funciona

↑ Índice

## La idea en una frase

Tu proyecto vive en tu PC. Lo subes a **GitHub**, conectas ese repositorio a **Vercel**, y Vercel ejecuta `npm run build` por ti y publica la carpeta resultante (`dist`) en una URL pública. Desde ese momento, cada `git push` actualiza el sitio solo.

**1. Código** src/, package.json en tu PC → **2. git push** el código llega a GitHub → **3. Build** Vercel corre `npm run build` → **4. Publica** sube `dist/` a su red global → **5. URL** mi-app .vercel.app

## Lo que debes tener instalado y creado

| Herramienta | Para qué sirve | Dónde conseguirla |
|---|---|---|
| **Node.js** versión 20.19+ o 22.12+ | Ejecuta Vite y npm. Las versiones actuales de Vite exigen estas versiones mínimas. | nodejs.org → descarga la versión **LTS** |
| **Git** | Guarda versiones de tu código y lo envía a GitHub. | git-scm.com/downloads |
| **Cuenta de GitHub** | Aloja tu repositorio. Vercel lo lee desde allí. | github.com/signup |
| **Cuenta de Vercel** | Compila y publica el sitio. El plan **Hobby** es gratis para proyectos personales. | vercel.com/signup (la creamos en el Paso 8) |
| **Editor de código** | Recomendado: Visual Studio Code. | code.visualstudio.com |

**IMPORTANTE PARA WINDOWS**

Al instalar Node.js y Git deja las opciones por defecto. Cuando termines, **cierra y vuelve a abrir** la terminal (o VS Code) para que reconozca los comandos nuevos.

## Dos conceptos que vas a ver en toda la guía

**BUILD (COMPILACIÓN)**

Vite convierte tus archivos `.tsx` en HTML, CSS y JavaScript optimizados dentro de la carpeta `dist/`. Eso, y no tu código fuente, es lo que se publica.

**DEPLOY (DESPLIEGUE)**

Es subir ese resultado a un servidor público. Vercel hace build + deploy automáticamente cada vez que subes cambios a GitHub.

FASE 1 · PASO 01

# Verificar Node.js, npm y Git

↑ Índice

Antes de todo, confirma que las herramientas están instaladas. Abre una terminal: en Windows, **PowerShell** o la terminal de VS Code (`Ctrl + ñ` o menú *Terminal → New Terminal*); en macOS/Linux, la app **Terminal**.

### 1 Escribe estos tres comandos, uno por uno, y presiona Enter

```
node -v
npm -v
git --version
```

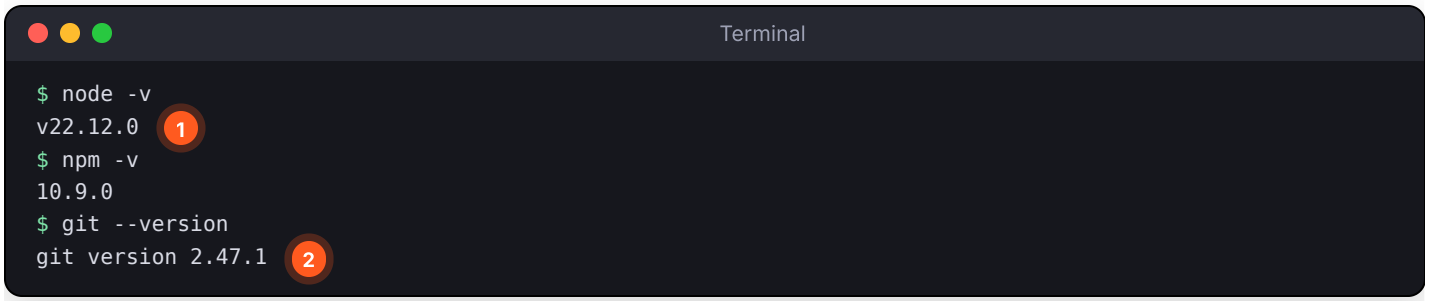



**Figura 1.** Respuesta esperada (tus números pueden ser distintos).

1. Node debe ser **20.19 o superior**, o **22.12 o superior**. Si ves `v18…` o `v16…`, instala la versión LTS desde nodejs.org.
2. Si Git responde con una versión, está listo. Cualquier versión reciente sirve.

**SI APARECE "NO SE RECONOCE COMO UN COMANDO"**

La herramienta no está instalada o la terminal se abrió antes de instalarla. Instálala, **cierra todas las terminales** y vuelve a probar. En casos raros, reinicia el equipo.

### 2 Configura tu identidad en Git (solo la primera vez en este equipo)

Git firma cada cambio con un nombre y un correo. Usa el mismo correo de tu cuenta de GitHub.

```
git config --global user.name "Tu Nombre"
git config --global user.email "tucorreo@ejemplo.com"
```

**LISTO CUANDO...**

Los tres comandos muestran una versión y configuraste nombre y correo en Git. Sigue al Paso 2 (o al Paso 4 si ya tienes proyecto).

FASE 1 · PASO 02

# Crear el proyecto con Vite

↑ Índice

Si ya tienes tu proyecto React + Vite + TypeScript, salta al Paso 3. Si no, aquí creas uno nuevo con la plantilla oficial.

## 1 Ubícate en una carpeta y crea el proyecto con la plantilla `react-ts`

```
cd Documents   # o la carpeta que prefieras
npm create vite@latest mi-proyecto -- --template react-ts
```

Cambia `mi-proyecto` por el nombre que quieras, en minúsculas y sin espacios. Los dos guiones `--` antes de `--template` son obligatorios: sin ellos npm ignora la plantilla.

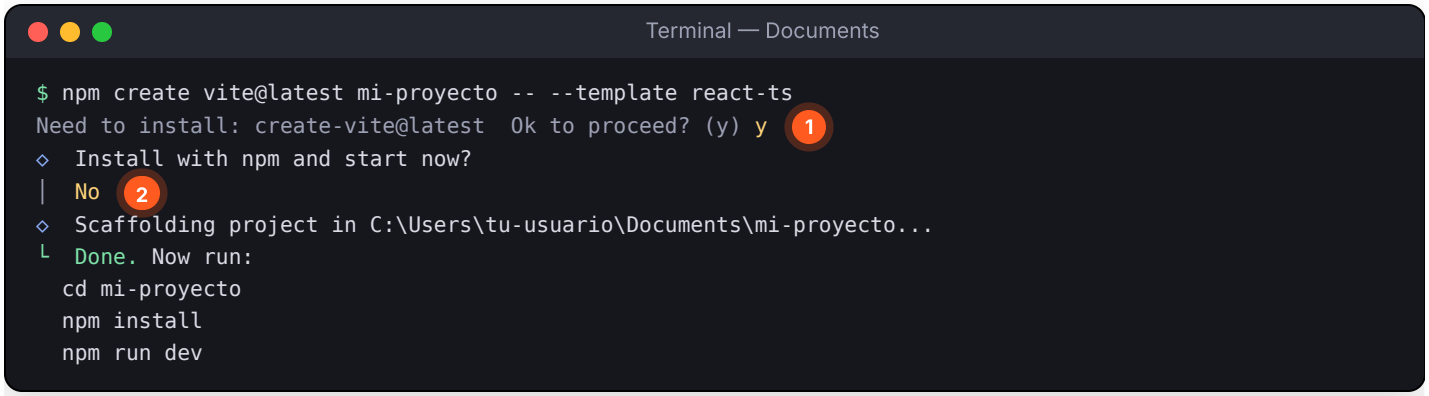



**Figura 2.** Creación del proyecto. Las preguntas pueden variar un poco según la versión de create-vite.

1. La primera vez npm pide permiso para descargar `create-vite`. Escribe `y` y Enter.
2. Si pregunta por instalar y arrancar ya, elige **No** para seguir esta guía al pie de la letra. Si pregunta por un linter, deja la opción por defecto.

### Lo que se crea

```
mi-proyecto/
├── public/              archivos estáticos
├── src/
│   ├── assets/
│   ├── App.tsx          tu componente principal
│   ├── App.css
│   ├── index.css
│   └── main.tsx         punto de entrada
├── .gitignore           lo que Git ignora
├── index.html
├── package.json         scripts y dependencias
├── tsconfig.json
├── tsconfig.app.json
├── tsconfig.node.json
└── vite.config.ts       configuración de Vite
```

**LOS 3 SCRIPTS QUE IMPORTAN**

Abre `package.json` y verás algo así:

```
"scripts": {
  "dev": "vite",
  "build": "tsc -b && vite build",
  "preview": "vite preview"
}
```

**dev** = trabajar en local · **build** = revisar tipos y compilar (Vercel usa este) · **preview** = probar lo compilado.

FASE 1 · PASO 03

# Ejecutarlo en local

↑ Índice

## 1 Entra a la carpeta e instala las dependencias

```
cd mi-proyecto
npm install
```

Esto crea la carpeta `node_modules/`. Puede tardar uno o dos minutos. **Nunca** se sube a GitHub.

## 2 Arranca el servidor de desarrollo

```
npm run dev
```

Mientras esté encendido, cada vez que guardas un archivo el navegador se actualiza solo. Para detenerlo: `Ctrl + C` en la terminal.

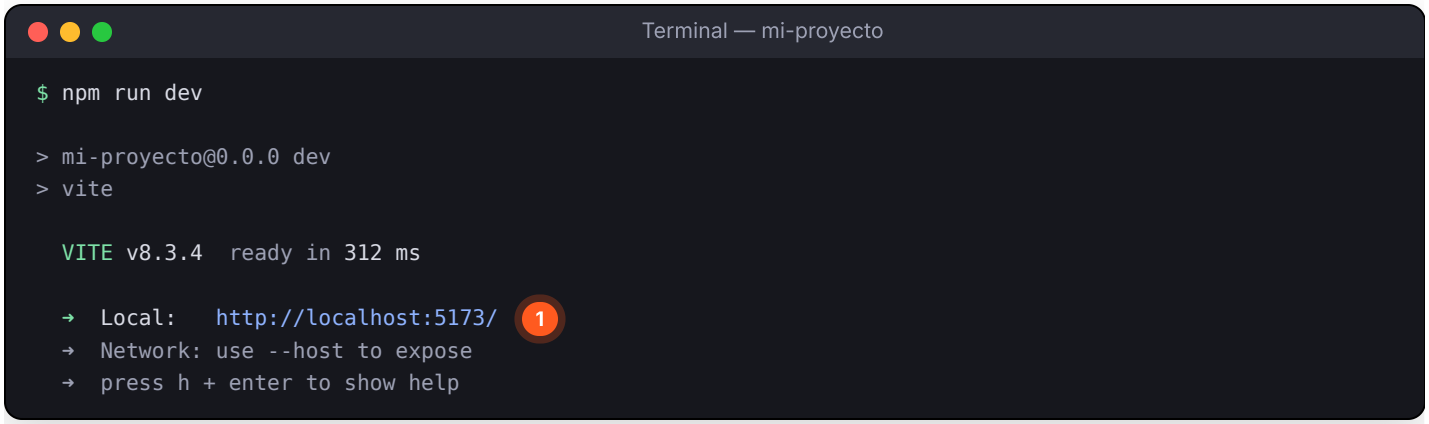



**Figura 3.** Servidor de desarrollo corriendo.

## 3 Abre la dirección en el navegador

1

Haz `Ctrl + clic` sobre `http://localhost:5173/` o cópiala en el navegador.

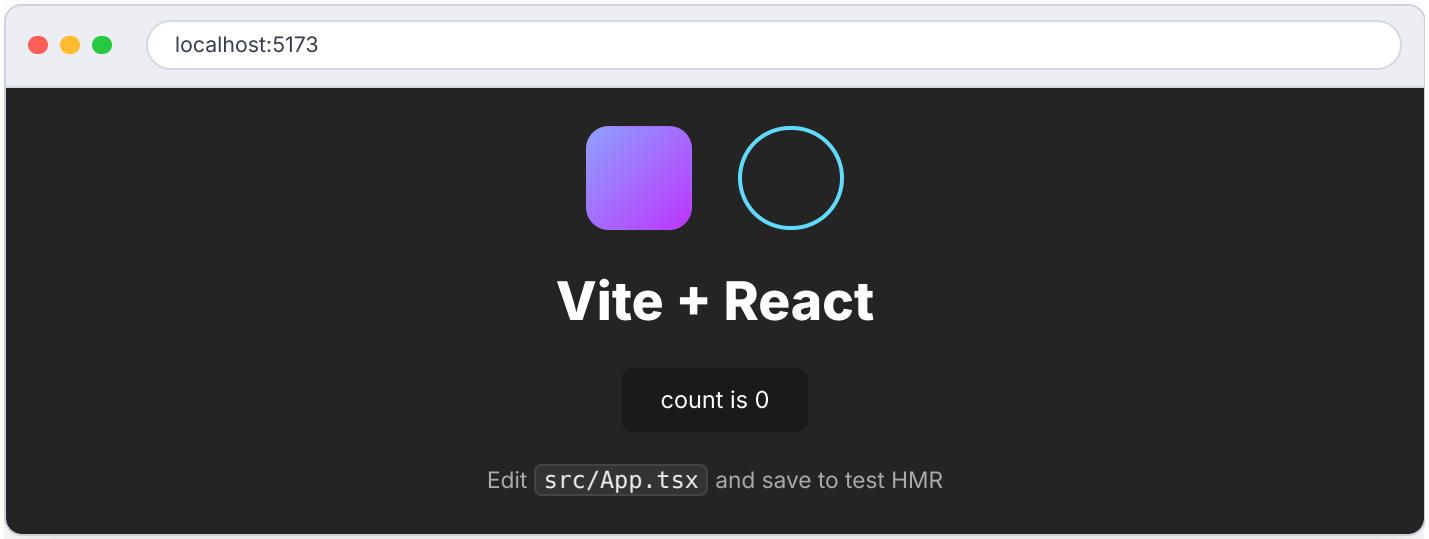



**Figura 4.** Página inicial de la plantilla (ilustración). Si ves tu proyecto funcionando, vas bien.

FASE 1 · PASO 04

# Probar el build de producción

↑ Índice

Este es el paso que más se olvida y el que más despliegues fallidos evita. Vercel va a ejecutar **exactamente** `npm run build`. Si falla en tu PC, fallará en Vercel.

## 1 Detén el servidor de desarrollo (`Ctrl + C`) y ejecuta el build

```
npm run build
```

✔ Build correcto

```
$ npm run build
> tsc -b && vite build

vite v8.3.4 building for production...
✓ 32 modules transformed.
dist/index.html          0.46 kB
dist/assets/index-B1x.css 1.39 kB
dist/assets/index-Cq2.js 195.2 kB
✓ built in 1.21s  (1)
```

**Figura 5a.** Así se ve cuando todo está bien.

✘ Build con error

```
$ npm run build
> tsc -b && vite build

src/App.tsx:4:10 - error TS6133:
'useEffect' is declared but its
value is never read.  (2)

4 import { useState, useEffect }
           ~~~~~~~~~

Found 1 error.
```

**Figura 5b.** Error de TypeScript: el build se detiene.

1. Se creó la carpeta `dist/` con el sitio listo. Esa carpeta es la que Vercel publicará (no la subes tú; Vercel la genera).
2. El error indica **archivo : línea : columna**. Corrígelo (aquí, borrar el import que no se usa), guarda y vuelve a ejecutar `npm run build` hasta que pase.

**¿POR QUÉ EN DEV FUNCIONA Y EN BUILD NO?**

`npm run dev` no revisa los tipos de TypeScript, solo muestra la página. `npm run build` ejecuta primero `tsc -b`, que es estricto: variables sin usar, tipos incorrectos o props faltantes detienen el proceso. Corrige todo en local antes de subir.

## 2 Prueba el resultado compilado

```
npm run preview
```

Abre `http://localhost:4173/`. Lo que ves aquí es casi idéntico a lo que verá la gente en Vercel. Navega por todas tus páginas y revisa la consola del navegador (`F12`) por errores.

**LISTO CUANDO...**

`npm run build` termina con ✔ **built in...** y `npm run preview` muestra tu sitio sin errores.

FASE 1 · PASO 05

# Dejar el proyecto listo para desplegar

↑ Índice

Cuatro revisiones rápidas. Las tres últimas solo aplican si tu proyecto usa esas cosas, pero léelas: son la causa de la mayoría de problemas después de desplegar.

### 5.1 · Revisa el `.gitignore`

Debe excluir dependencias, compilados y archivos con secretos. La plantilla de Vite ya trae casi todo; asegúrate de que tenga al menos estas líneas:

.gitignore

```
node_modules
dist
*.local
.env
```

### 5.2 · ¿Usas React Router? Agrega `vercel.json`

Si tu app tiene rutas como `/login` o `/productos/3`, al recargar la página en Vercel verás un **404**, porque el servidor busca un archivo con ese nombre. Esta regla le dice que siempre entregue `index.html` y deje que React Router decida qué mostrar. Crea el archivo en la **raíz** del proyecto (al lado de `package.json`):

vercel.json

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

Si tu proyecto tiene una sola página (sin rutas), no lo necesitas.

### 5.3 · ¿Usas variables de entorno? Usa el prefijo `VITE_`

Vite solo expone al navegador las variables que empiezan por `VITE_`. En local van en un archivo `.env`:

.env

```
VITE_API_URL=https://mi-api.com
```

Y se leen así en el código:

```
const url = import.meta.env.VITE_API_URL;
```

**EL .ENV NO VIAJA A VERCEL**

Como está en `.gitignore`, Vercel no lo ve. Tendrás que escribir las mismas variables en el panel de Vercel (lo hacemos en el Paso 10).

**NADA SECRETO EN EL FRONTEND**

Todo lo que tenga `VITE_` queda visible en el JavaScript público. No pongas contraseñas de bases de datos ni llaves privadas.

### 5.4 · Revisa mayúsculas y minúsculas en los imports

Windows no distingue `Navbar.tsx` de `navbar.tsx`, pero los servidores de Vercel (Linux) **sí**. Un import mal escrito funciona en tu PC y rompe el build en Vercel con un error tipo *Could not resolve*.

```
// archivo real: src/components/Navbar.tsx
✘ import Navbar from "./components/navbar";
✔ import Navbar from "./components/Navbar";
```

FASE 2 · PASO 06

# Crear el repositorio en GitHub

↑ Índice

Un repositorio es la “carpeta en la nube” donde vivirá tu código. Vercel lo leerá desde allí.

**1** **Inicia sesión en github.com y crea un repositorio nuevo**

Arriba a la derecha toca el botón **+** → **New repository**. También puedes ir directo a github.com/new.

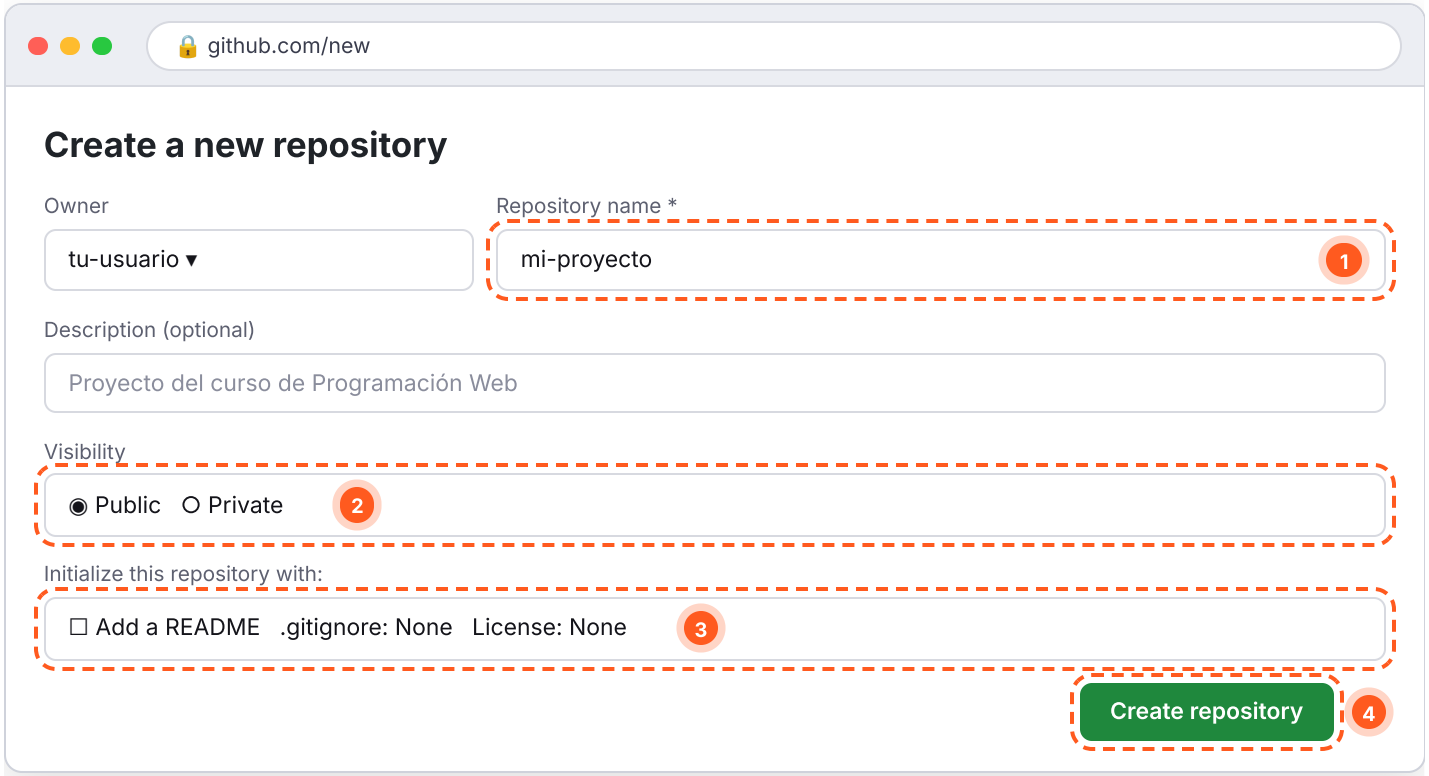



**Figura 6.** Formulario de nuevo repositorio en GitHub (ilustración).

1. **Nombre:** idealmente el mismo de tu carpeta, en minúsculas y con guiones.
2. **Public o Private:** ambos funcionan con Vercel. Private si no quieres que otros vean el código.
3. **No marques nada aquí** (ni README, ni .gitignore, ni licencia). Tu proyecto local ya tiene archivos; si GitHub crea otros, el primer `push` será rechazado.
4. Toca **Create repository**. GitHub te mostrará una página con comandos; copia la URL que termina en `.git`.




**Figura 7.** URL del repositorio vacío. 5 Cópiala con el botón de copiar: la usarás en el siguiente paso.

FASE 2 · PASO 07

# Subir el código con Git

↑ Índice

En la terminal, **dentro de la carpeta del proyecto**, ejecuta estos comandos en orden. Cambia la URL por la que copiaste.

```
# 1. Convierte la carpeta en un repositorio Git
git init

# 2. Prepara todos los archivos (respetando el .gitignore)
git add .

# 3. Guarda una versión con un mensaje
git commit -m "Primer commit: proyecto React + Vite + TypeScript"

# 4. Nombra la rama principal como main
git branch -M main

# 5. Conecta tu carpeta con el repositorio de GitHub
git remote add origin https://github.com/tu-usuario/mi-proyecto.git

# 6. Sube el código
git push -u origin main
```

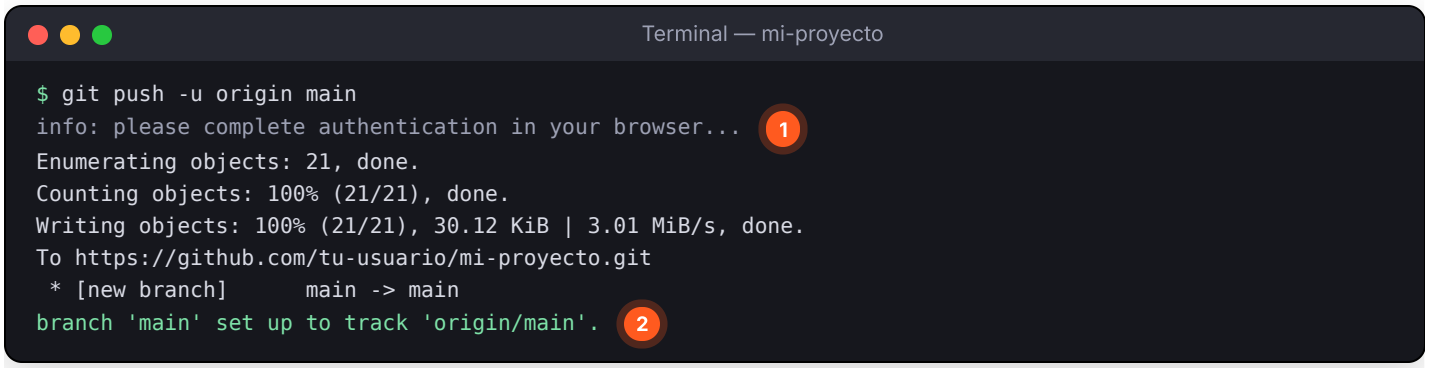



**Figura 8.** Primer push exitoso.

1. La primera vez se abre el navegador para que autorices el acceso a GitHub. Inicia sesión y acepta. GitHub ya no acepta tu contraseña normal desde la terminal.
2. Recarga la página del repositorio en GitHub: deberías ver tus carpetas `src`, `public` y demás archivos. **No** debe aparecer `node_modules`.

### ALTERNATIVA SIN COMANDOS

En VS Code abre la pestaña **Source Control** (icono de ramas, `Ctrl + Shift + G`) y usa **Publish to GitHub**. Hace los pasos 1 a 6 por ti y crea el repositorio.

### SI EL PUSH ES RECHAZADO

**rejected / fetch first:** marcaste README al crear el repo. Ejecuta `git pull origin main --allow-unrelated-histories` y vuelve a hacer push. **remote origin already exists:** usa `git remote set-url origin URL`.

FASE 3 · PASO 08

# Crear la cuenta en Vercel

↑ Índice

Lo más práctico es registrarte **con tu cuenta de GitHub**: así Vercel puede ver tus repositorios sin configuraciones extra.

1 **Entra a [vercel.com/signup](vercel.com/signup)**

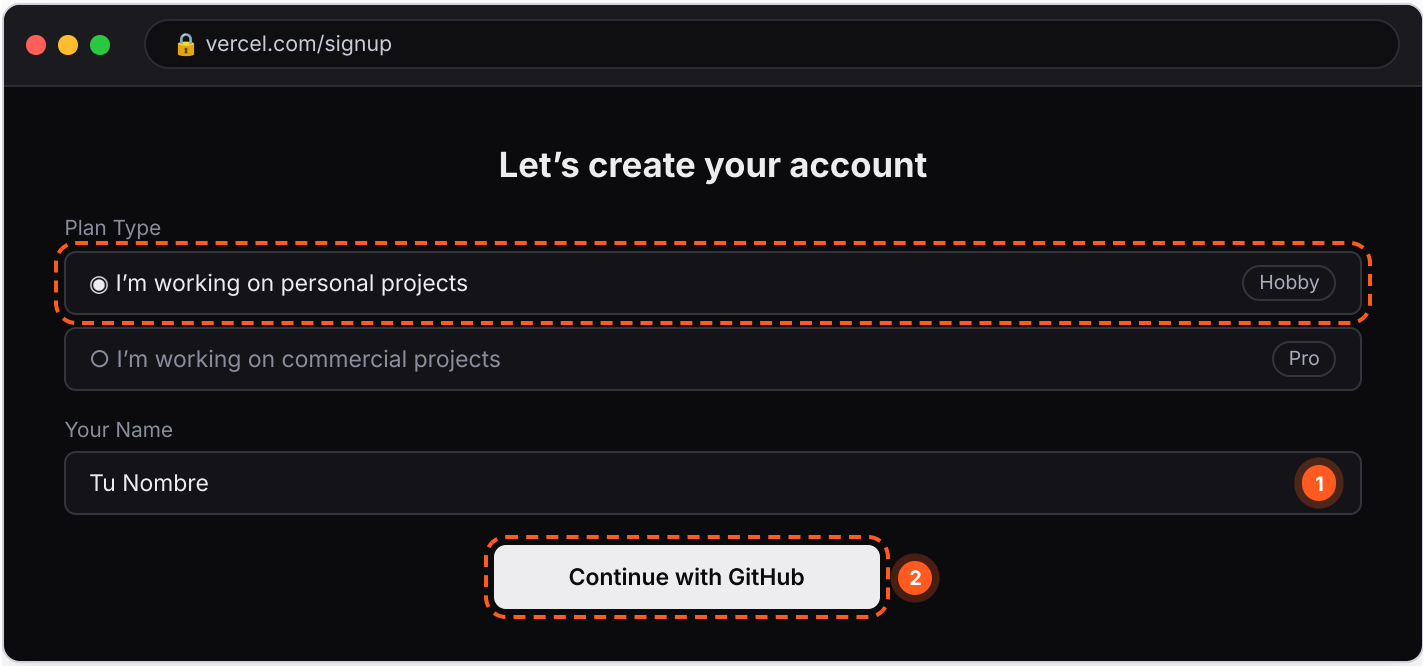



**Figura 9.** Registro en Vercel (ilustración; la pantalla real puede pedir los datos en otro orden).

1. Elige **Hobby** (personal): es gratis y suficiente para proyectos académicos. Escribe tu nombre.
2. Toca **Continue with GitHub**. GitHub te pedirá autorizar a Vercel: toca **Authorize Vercel**. Puede pedirte verificar un número de teléfono.

**¿YA TIENES CUENTA DE VERCEL CON CORREO?**

Puedes conectar GitHub después desde **Account Settings → Authentication** (o al importar tu primer repositorio, Vercel te lo pedirá).

**LISTO CUANDO...**

Ves el panel principal (dashboard) de Vercel con tu nombre arriba a la izquierda.

FASE 3 · PASO 09

# Importar el repositorio

↑ Índice

1. **En el dashboard toca Add New... → Project**
   También puedes ir directo a vercel.com/new.

2. **Busca tu repositorio en la lista y toca Import**

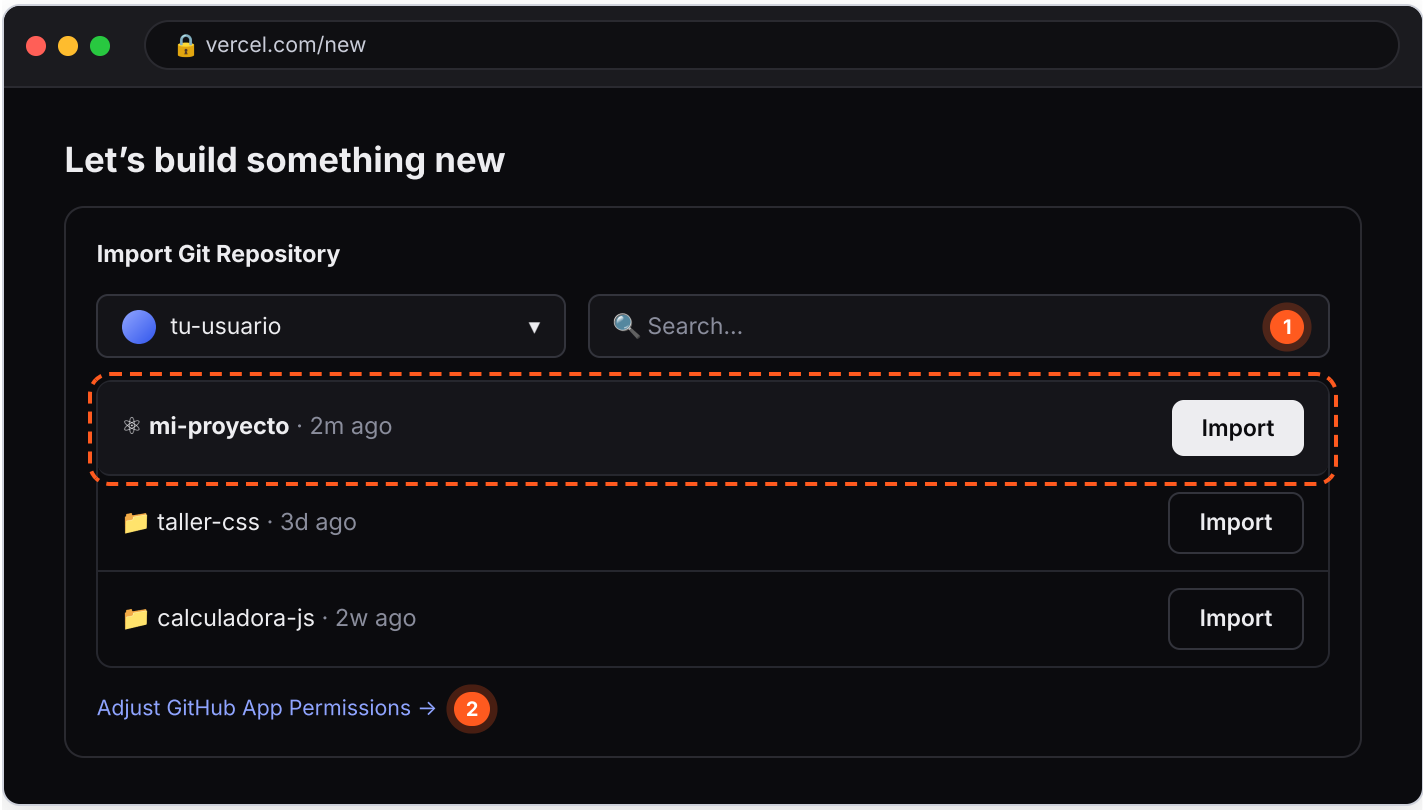



**Figura 10.** Lista de repositorios de GitHub dentro de Vercel (ilustración). Toca **Import** en el tuyo.

1. Si tienes muchos repositorios, escribe el nombre en el buscador.
2. **¿No aparece tu repositorio?** Toca este enlace (o *Configure GitHub App*). En GitHub elige **All repositories** o selecciona `mi-proyecto` en *Only select repositories*, guarda con **Save** y vuelve a Vercel.

**PRIMERA VEZ**

Si es la primera importación, Vercel te pide **instalar la aplicación de Vercel en GitHub**. Acepta: es lo que le permite leer el repositorio y desplegar en cada `push`.

FASE 3 · PASO 10

# Configurar y desplegar

↑ Índice

Vercel detecta que es un proyecto de Vite y llena casi todo solo. Tu trabajo es **verificar** estos campos y tocar **Deploy** 6.

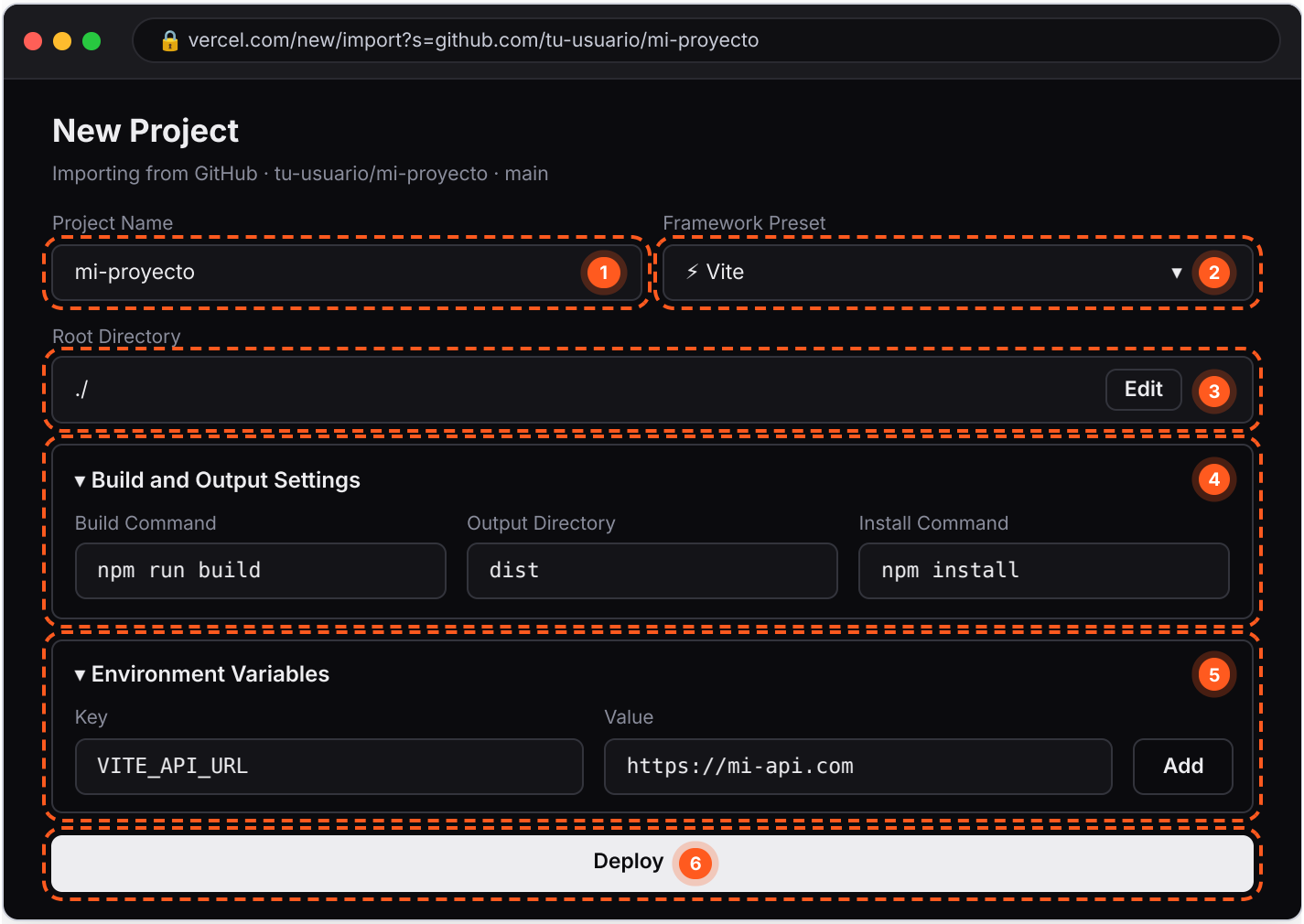



**Figura 11.** Pantalla de configuración del proyecto (ilustración).

| # | Campo | Valor correcto | Qué revisar |
|---|---|---|---|
| 1 | **Project Name** | `mi-proyecto` | Define tu URL: `mi-proyecto.vercel.app`. Si el nombre ya existe, Vercel agrega un sufijo. |
| 2 | **Framework Preset** | `Vite` | Debe detectarlo solo. Si dice *Other*, cámbialo a **Vite**. |
| 3 | **Root Directory** | `./` | Déjalo así si `package.json` está en la raíz del repo. Si tu proyecto está en una subcarpeta (ej. `frontend/`), toca **Edit** y elígela. |
| 4 | **Build and Output** | `npm run build`<br>`dist` | Los valores del preset de Vite son correctos. No los cambies salvo que sepas por qué. |
| 5 | **Environment Variables** | las de tu `.env` | Solo si usas variables. Copia cada línea del `.env`: nombre en *Key*, valor en *Value*, toca **Add**. |

FASE 3 · PASO 11

# Ver tu sitio publicado

↑ Índice

## 1 Observa los registros del build

Mientras despliega verás el log en vivo. Son los mismos mensajes que viste en tu PC con `npm run build`. Suele tardar menos de un minuto.




**Figura 12.** Registro de un build exitoso en Vercel (ilustración).

1. Vercel instala dependencias desde cero con tu `package.json`. Por eso no subes `node_modules`.
2. Si en vez de esto ves texto en rojo y **Build Failed**, lee la primera línea de error y ve a Errores comunes.

## 2 Celebra y abre tu URL

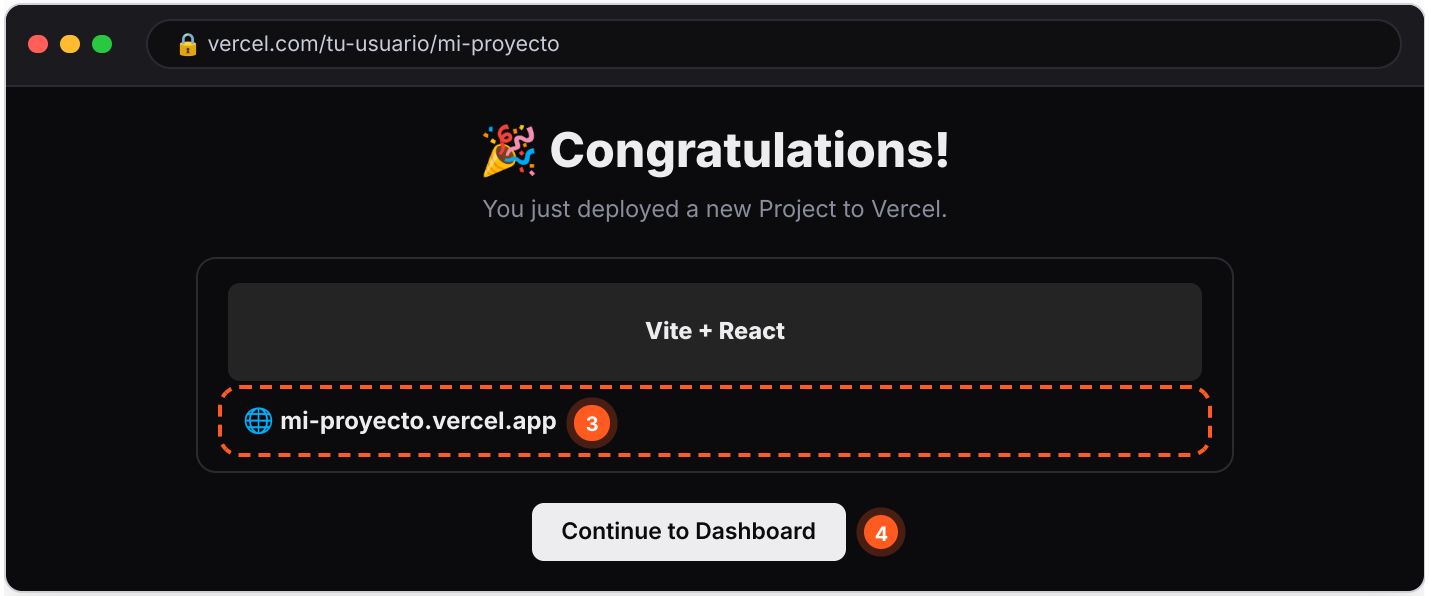



**Figura 13.** Pantalla de despliegue exitoso (ilustración).

3. Toca la vista previa o la URL para abrir tu sitio. **Esa es la dirección que compartes** con el profesor o tus compañeros.
4. En el dashboard del proyecto encuentras la URL de producción, el historial de despliegues (**Deployments**) y la configuración (**Settings**).

**PRUEBA FINAL**

Abre la URL desde tu celular con datos móviles. Si usas rutas, entra a una y **recarga**: no debe salir 404.

FASE 3 · PASO 12

# Actualizar el sitio con cada cambio

↑ Índice

No tienes que volver a entrar a Vercel. Cada vez que subes cambios a GitHub, Vercel vuelve a compilar y publica solo. Esto se llama **despliegue continuo**.

**Editas** cambias App.tsx → **Pruebas** `npm run build` → **Commit** `git commit` → **Push** `git push` → **Vercel** redespliega solo en ~1 min

## El ciclo de todos los días

```
npm run build                          # 1. verifica que compila
git add .                              # 2. prepara los cambios
git commit -m "Agrega página de contacto"  # 3. guarda la versión
git push                               # 4. sube → Vercel despliega
```

## Producción y vistas previas

**RAMA MAIN → PRODUCCIÓN**

Lo que subes a `main` actualiza la URL principal `mi-proyecto.vercel.app`.

**OTRAS RAMAS → PREVIEW**

Si subes una rama distinta (ej. `git push origin nueva-funcion`), Vercel crea una URL de **vista previa** aparte, sin tocar producción. Ideal para trabajar en grupo y revisar antes de unir.

## Cosas útiles en el dashboard

| Quiero... | Dónde |
|---|---|
| Volver a una versión anterior que funcionaba | **Deployments** → menú ⋯ del despliegue bueno → **Promote to Production** (o *Instant Rollback*). |
| Agregar o cambiar variables de entorno | **Settings → Environment Variables**. Después **vuelve a desplegar** (Deployments → ⋯ → **Redeploy**): los cambios no se aplican al despliegue que ya existe. |
| Cambiar el nombre de la URL | **Settings → Domains** → edita el dominio `.vercel.app` o agrega un dominio propio. |
| Cambiar la versión de Node.js | **Settings → Build and Deployment → Node.js Version**. Usa la misma versión mayor que en tu PC. |
| Agregar a compañeros del grupo | En el plan Hobby no hay equipos. Lo más simple: agrégalos como colaboradores **del repositorio en GitHub** (Settings → Collaborators); sus pushes también disparan despliegues. |

EXTRA A

# Alternativa: desplegar con Vercel CLI

↑ Índice

Si no quieres usar GitHub (o quieres desplegar rápido desde la terminal), Vercel tiene una herramienta de línea de comandos. Útil para pruebas, pero **no actualiza solo** al hacer cambios: debes repetir el comando.

### 1 Instala la CLI de forma global

```
npm install -g vercel
```

### 2 Inicia sesión

(abre el navegador para confirmar)

```
vercel login
```

### 3 Desde la carpeta del proyecto, despliega

```
vercel
```




**Figura 14.** Primer despliegue con la CLI (las preguntas pueden variar según la versión).

1. Igual que en el panel web, detecta Vite y usa `npm run build` y `dist`. Responde **no** para dejarlos así.
2. El comando `vercel` crea una URL de **vista previa**. Para publicar en la URL principal usa el paso 4.

### 4 Publica en producción

```
vercel --prod
```

**¿PANEL WEB O CLI?**

Para el curso, recomendamos **GitHub + panel web** (Pasos 6 a 12): queda historial, el despliegue es automático y el profesor puede ver el código. La CLI es un atajo.

EXTRA B

# Errores comunes y cómo solucionarlos

↑ Índice

Regla de oro: cuando falla un build en Vercel, abre el despliegue, ve a **Build Logs** y busca la **primera** línea en rojo. Casi siempre es una de estas.

| Síntoma | Causa probable | Solución |
|---|---|---|
| **Build Failed** con `error TS…` | Errores de TypeScript que `npm run dev` no mostraba. | Ejecuta `npm run build` en tu PC, corrige cada error, haz commit y push. Ver Paso 4. |
| **Could not resolve "./components/navbar"** | Mayúsculas/minúsculas distintas entre el import y el archivo. | Escribe el import exactamente igual al nombre del archivo. Ver 5.4.<br>Si renombraste solo mayúsculas, Git puede no notarlo: usa `git mv navbar.tsx Navbar.tsx`. |
| **404: NOT_FOUND** al recargar una ruta como `/login` | Falta la regla para aplicaciones de una sola página (SPA). | Crea `vercel.json` con el *rewrite* a `/index.html`. Ver 5.2. |
| **Página en blanco**; en consola (F12) errores 404 de archivos `.js` o `.css` | Se cambió `base` en `vite.config.ts` (p. ej. para GitHub Pages). | En Vercel no hace falta `base`. Quítalo o déjalo como `base: '/'`. |
| **Página en blanco** sin errores de archivos | Un error de JavaScript en tiempo de ejecución. | Abre la consola (F12) en la URL de Vercel y lee el error. Prueba con `npm run preview` en local para reproducirlo. |
| La variable sale **undefined** en producción | No tiene prefijo `VITE_`, no se agregó en Vercel o no se redesplegó. | Renómbrala a `VITE_…`, agrégala en **Settings → Environment Variables** y haz **Redeploy**. |
| **No such file: package.json** o no detecta Vite | El proyecto está en una subcarpeta del repositorio. | Settings → Build and Deployment → **Root Directory** → elige la carpeta que contiene `package.json`. |
| Mi repositorio **no aparece** en Vercel | La app de Vercel en GitHub no tiene acceso a ese repo. | En la pantalla de importar: **Adjust GitHub App Permissions** → da acceso al repo. Ver Paso 9. |
| **Unsupported engine** / error de versión de Node | Node muy viejo en tu PC o en Vercel. | Instala Node LTS en tu PC. En Vercel: Settings → Build and Deployment → **Node.js Version**. |
| **npm ERR! ERESOLVE** (conflicto de dependencias) | Paquetes con versiones incompatibles entre sí. | En tu PC, ejecuta `npm install`, resuelve el conflicto y sube el `package-lock.json` actualizado. Siempre sube ese archivo. |
| Hice push y **no se actualizó** | El push fue a otra rama, o el build falló. | Revisa la pestaña **Deployments**: ¿hay uno nuevo? ¿dice *Error* o *Preview*? Producción solo se actualiza desde `main`. |
| Se subió **node_modules** a GitHub | Faltaba en `.gitignore` al hacer el primer commit. | Agrégalo al `.gitignore` y ejecuta `git rm -r --cached node_modules`, luego commit y push. |

EXTRA C

# Resumen de comandos y checklist final

↑ Índice

## Todos los comandos, en orden

```
# Verificar herramientas
node -v && npm -v && git --version

# Crear el proyecto (solo desde cero)
npm create vite@latest mi-proyecto -- --template
react-ts
cd mi-proyecto
npm install

# Trabajar y probar
npm run dev        # localhost:5173
npm run build      # debe terminar en ✓
npm run preview    # localhost:4173

# Subir a GitHub (primera vez)
git init
git add .
git commit -m "Primer commit"
git branch -M main
git remote add origin URL_DEL_REPO
git push -u origin main

# Actualizar (cada cambio)
git add .
git commit -m "Describe el cambio"
git push
```

## Checklist antes de entregar

- [ ] Node 20.19+ o 22.12+ instalado
- [ ] `npm run build` termina sin errores en mi PC
- [ ] `.gitignore` incluye `node_modules`, `dist` y `.env`
- [ ] Si uso React Router: creé `vercel.json`
- [ ] Imports con mayúsculas exactas
- [ ] Variables con prefijo `VITE_` y sin secretos
- [ ] Código en GitHub, sin `node_modules`
- [ ] Proyecto importado en Vercel con preset **Vite**
- [ ] Variables de entorno cargadas en Vercel
- [ ] La URL `.vercel.app` abre desde el celular
- [ ] Recargar una ruta interna no da 404
- [ ] Un `git push` nuevo actualiza el sitio

## Valores de Vercel para Vite

| | |
|---|---|
| **Framework** | `Vite` |
| **Build** | `npm run build` |
| **Output** | `dist` |
| **Install** | `npm install` |
| **Root** | `./` |

## Enlaces útiles

Guía oficial de Vite
Variables de entorno en Vite
Vite en la documentación de Vercel
Configuración de vercel.json
Documentación de Vercel CLI
Primeros pasos con GitHub (español)